

```

soundset          = {point to the
sound you want played around the
decal.             For example,
audiobank\mysounds\mydecal.wav}

```

```

;identity details
strength          = 0
isimmobile        = 1
ismarker          = 4

```

```

;decals
decamax           = 1
decal0            = {give your decal
a name}

```

In the gamecore\decals folder, create a folder with the *exact* same name that you gave for decal0 above. A quick solution now is to copy the files from one of the other decals and paste them here, and then edit them. Don't forget the Alpha channel!

This was just decals—kid stuff when you compare it to what else you can do to the look of the game. You can also modify textures (they're in the texturebank folder) the same way, and with some knowledge of 3D, even add new guns!

Get Smart

FPS Creator uses its own FPI scripting language for all the AI in your games, and learning it is quite intuitive. It hides the more low-level AI away from you, but gives you control over the actions that entities take when faced with a condition. You'll find these scripts in the scriptbank folder, and they'll open in either Notepad or WordPad. Your typical FPI line looks like this:

```

:condition1,
(condition2)...: action1,
(action2)....

```

The literal translation would be thus: If condition1, condition2, etc. are all satisfied, perform action1, action2, and the rest. You will find the whole list of conditions and actions in the FPS Creator manual—there are an obscene number of them to choose from. Let's start with a basic tutorial—a light that turns on when you're near it, and off when you go far. In the scriptbank\user folder, create a new script called myscript.fpi, open it in Notepad, and add this:

```

;Artificial Intelligence Script

;Header

desc      = proximity light

;Triggers

:state=0,plrdistfurther=100:lig
htoff=1,state=1
:state=1,plrdistwithin=100:ligh
ton=1,state=0

;End of Script

```

The lines that begin with a semicolon are comments, and standard to every script. The "state" variable is what FPS Creator uses to distinguish between different states of the entity; otherwise all the actions would be taking place at once. When you're writing more complex scripts, always make sure you have distinct states for each set of actions—ignoring them could cause your game to crash. Now to use this script in your game.

Start a new project in FPS creator, put in a prefab, and from the Markers, choose one of the lights. Place it anywhere in the scene. Now right-click on it, and you'll be taken to an editing screen which shows you a 3D view of the light, and a configuration panel on the left. In the configuration, change Static Mode to No, and under AI Scripts, point the Main script to the one you just created. Click Apply Changes.

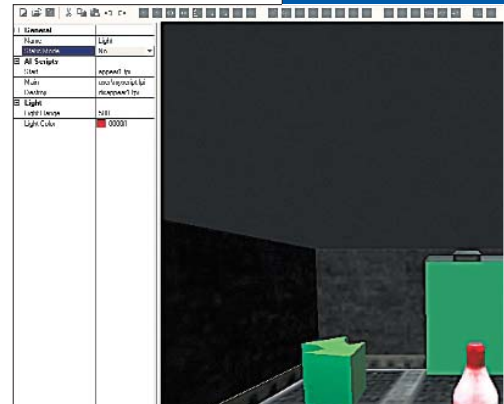
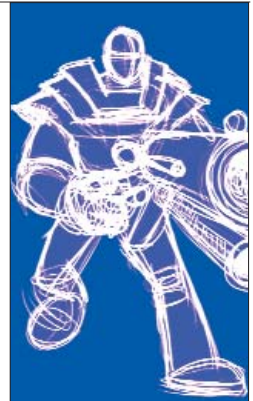
When you run the game and walk towards the light now, it should turn on in response to your proximity. Remember, you won't see a 3D model for the light—it will appear as if from nowhere.

This is very basic behaviour, but as you can see, FPI scripting is quite simple and the best way to understand more is to simply read the scripts in the script bank and keep the manual handy.

Boss Battle

If we were to demystify FPS Creator for you, we'd probably end up filling up most of this issue. However, the developers have made it easy for us—the files in FPS Creator are organised so as to be easy to fathom for the inquisitive mind: getting used to the scripts and other files took less than a week for us. In essence, if you've got the itch, the scratch isn't far. ■

nimish_chandiramani@thinkdigit.com



The configuration panel on the left lets you customise your entity before you hack the AI scripts

The before (above) and after (left) picture; we've used the chest of drawers as a reference for distance